

Assignment 1

Breakout Project Report

Interactive Systems in Processing (UI/UX)

Prepared 6 October 2026 | Current sketch documentation

Group members: _____

Student IDs / course / section: _____

Project overview

Breakout is an interactive paddle-and-ball game built in Processing Java mode. The player launches a ball, moves a paddle to keep it on screen, and clears eight stationary bricks. Each destroyed brick earns 10 points. Clearing every brick wins the game; missing the ball three times ends it.

This report documents the original 100-pixel-wide paddle sketch created in this conversation. The smaller paddle and improved keyboard handling discussed later are proposed changes, not confirmed changes in this version.

Assignment requirements: important fit check

The supplied instructions ask students to choose one of five listed projects. Breakout is not explicitly listed. Its closest match is Option 4, Physics-Based Puzzle Game, but the current sketch does not meet all of that option's requirements. This report does not claim full compliance.

Option 4 requirement	Current implementation
Gravity and collisions	Wall, paddle and brick collision logic is present. Gravity is absent; velocity stays constant between bounces.
Click/drag to move or place objects	Absent. The sketch uses keyboard controls.
Win condition: stack objects or clear shapes	Present: clear all eight bricks.
Levels that increase difficulty	Absent. There is one fixed brick layout.

Before submission, adapt the project to the missing requirements or confirm with the instructor that this Breakout variation is acceptable. The complete original assignment appears unchanged in the appendix.

How to run and use the project

1. Place **Breakout.pde** inside a folder named **Breakout**.
2. Open the sketch in the Processing IDE using Java mode.
3. Click Run. Move the paddle and press Space to launch.
4. Clear the bricks, or press R to start again. No image files or external libraries are required.

Control	Action
Left / right arrows	Move the paddle horizontally.
Space	Launch the ball when play has not ended.
R	Reset the paddle, ball, score, lives and bricks.

Arrays and loops

Three parallel arrays describe eight bricks. The same index identifies the X coordinate, Y coordinate and alive state of one brick. Index 0 identifies the brick at (90, 100).

```
float[] brickX = {90, 230, 370, 510, 90, 230, 370, 510};
float[] brickY = {100, 100, 100, 100, 145, 145, 145, 145};
boolean[] brickAlive = {true, true, true, true,
                        true, true, true, true};
```

A for loop checks each surviving brick and calls a reusable function to draw it. Other loops check collisions and restore bricks on restart.

```
for (int i = 0; i < brickAlive.length; i++) {
  if (brickAlive[i]) {
    bricksRemaining = true;
    drawBrick(brickX[i], brickY[i]);
  }
}
```

Functions and code organization

setup() creates a 600 x 500 window once. **draw()** repeatedly updates positions, checks collisions and draws the scene. **keyPressed()** handles launch and restart. **drawBrick(x, y)** draws a brick using its coordinates. **restartGame()** restores the initial game state. No custom classes are used in this version.

Animation, collisions and game rules

The top-left corner is (0, 0). Increasing X moves right; increasing Y moves down. The ball starts with speedX = 3 and speedY = -4, so it initially moves right and upward.

```
ballX = ballX + speedX;
ballY = ballY + speedY;

if (ballX <= 10) {
  ballX = 10;
  speedX = -speedX;
}
```

The ball has a radius of 10 pixels. At the left wall its centre is clamped to X = 10, and reversing horizontal speed makes it bounce right. Similar checks handle the right and top walls.

The paddle check detects a falling ball crossing Y = 430 and overlapping the paddle horizontally. It reverses vertical speed. Landing on the left or right half sets horizontal speed to -3 or 3 respectively.

Each brick is 120 x 30 pixels. Collision uses the ball's bounding square and the brick's rectangular bounds. A hit marks that brick false, adds 10 points, reverses vertical speed and exits the loop. Side hits also reverse vertical speed: this is simplified collision response.

```
brickAlive[i] = false;
score = score + 10;
speedY = -speedY;
break;
```

A ball entirely below the window removes one life and returns above the paddle. With no lives left, gameLost becomes true. When no living bricks remain, gameWon becomes true.

Example output visual

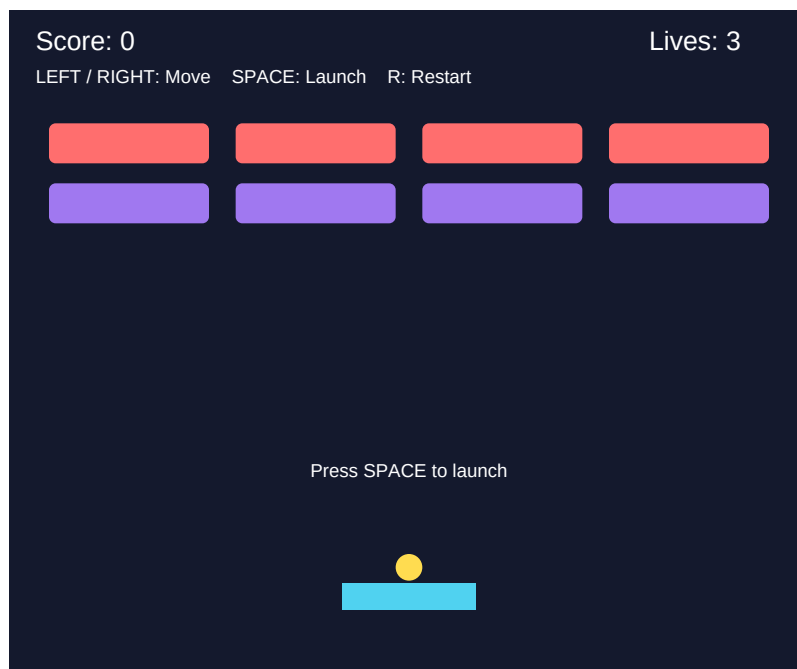


Figure 1. Reconstructed initial screen based on the drawing code. This illustration is not a screenshot from a tested Processing run.

Example end screen

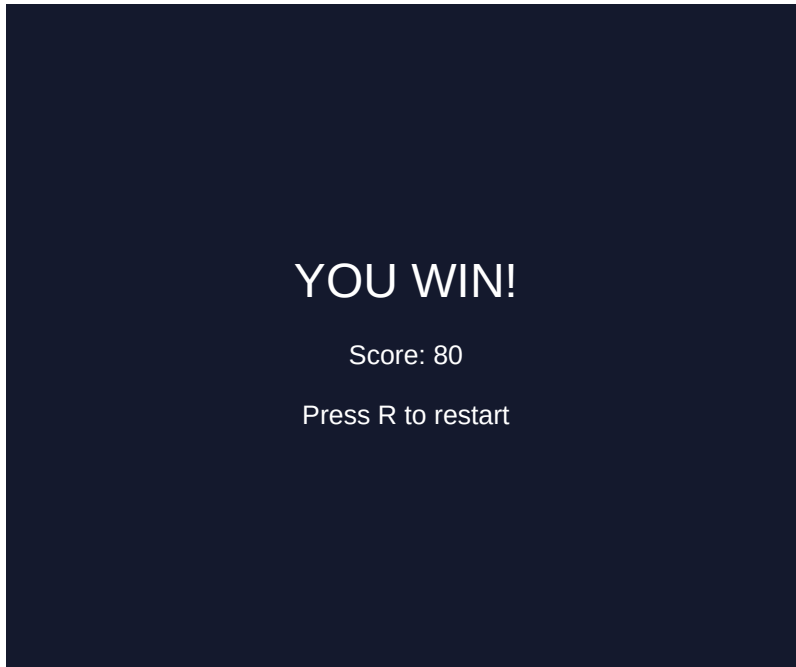


Figure 2. Reconstructed winning screen: eight bricks x 10 points = 80. Illustrative output, not runtime evidence.

Validation to complete in Processing

The sketch has not been run in Processing in this environment. Complete these checks and capture real screenshots before presenting this report as final test evidence.

Check	Expected result
Launch and paddle limits	Space launches; paddle stays inside both edges.
Walls and bricks	Ball bounces; each brick adds 10 points once.
Lives and result screens	A miss removes one life; zero lives loses; clearing eight bricks wins.
Restart and keyboard responsiveness	R restores all state. Check switching directions and overlapping arrow presses.

Known limitations and possible changes

Keyboard movement uses the built-in `keyPressed` state and one `keyCode`. Overlapping arrow presses may interrupt movement. Tracking each arrow separately is a proposed improvement. A narrower paddle is another proposed difficulty change; its draw width, collision half-width and screen limits must be updated together.

No gravity, mouse dragging or increasing levels are implemented. A standalone `README.md` is also required by the rubric; the run instructions above can be adapted into that file.

Demo explanations

1. What does the project do?

It is a Breakout game where the player bounces a ball to clear eight bricks, earn points and avoid losing three lives.

2. How does the user interact?

Arrow keys move the paddle, Space launches the ball and R restarts.

3. Where is a loop used?

For loops check brick collisions, draw surviving bricks and restore all bricks when restarting.

4. Where is an array used?

brickX, brickY and brickAlive store each brick's position and whether it remains.

5. Where is a function used?

drawBrick(x, y) draws a brick. restartGame() resets the game.

6. What did the group change or create?

Describe only changes your group actually made. The starting sketch was AI-assisted; group modifications and testing have not been supplied for this report.

Documentation and submission checklist

Complete the group identification fields; check project approval or implement the missing requirements; run the validation checks; capture actual output screenshots; prepare the separate README.md; and submit the Processing sketch with the documentation required by your instructor.

Original instructions included

The next five pages reproduce the supplied Assignment 1.pdf unchanged. They include the overview, all five project options, the 15-point marking rubric and the example README. The appendix is the authoritative source for the assignment wording.

Rubric: Functionality 5 points; code quality 2; creativity/design 2; performance/stability 3; documentation/README 3. Total: 15 points.

Assignment 1 — Interactive Systems in Processing

(UI/UX)

Overview

Build a small interactive program in **Processing** that demonstrates real-time graphics, animation, and user input. Choose **one** project from the list below and meet the **Project-Specific Requirements**.

What you'll practice

- Coordinate systems, animation loops, and timing
- Event handling (mouse/keyboard)
- Data structures (arrays/ArrayList)
- Functions and classes
- Simple physics or rule-based behaviors
- Clean code and documentation



AWESOME!!!!!!!!!!!!!!

Project Options: -

1. Interactive Storybook with Animations

- Concept: A digital storybook with multiple scenes. Each scene has characters (drawn objects) that move or animate.
- Features:
 - ≥ 3 scenes with transitions (fade, slide, wipe).
 - At least 2 animated characters per scene (tweening, frame animation).
 - User choices branch the story (if user clicks left $\rightarrow$ one scene, right $\rightarrow$ another).
 - Optional background music or sound effects.

2. Music-Responsive Visualizer

- Concept: An audio visualizer that reacts to microphone input or an audio file.
- Features:
 - Use amplitude (loudness) $\rightarrow$ bar height or circle size.
 - Use frequency spectrum $\rightarrow$ multiple shapes (bars, waves, radial pattern).
 - At least 2 different modes toggleable via keys.
 - History trail to show sound change over time.

3. Virtual Aquarium with Fish Behavior

- Concept: An animated aquarium simulation.
- Features:
 - ≥ 15 fish (objects) that wander with randomness + wall avoidance.
 - Clicking drops “food particles” that fish chase.
 - Fish avoid the mouse cursor (predator effect).
 - Extra: day/night toggle (background color change).
 - Extra: fish have a hunger/energy bar that affects movement.

4. Physics-Based Puzzle Game

- Concept: A simple game where the player interacts with falling blocks, balls, or shapes.
- Features:
 - Gravity and collisions (basic physics engine logic).
 - User can click/drag to move or place objects.
 - Win condition: stack objects to reach a goal or clear shapes.
 - Levels increase difficulty (different layouts or shapes).

5. Crowd & Flocking Simulator (Boids-style)

- Concept: A simulation of a crowd of agents (birds, people, cars).

- Features:
 - Agents follow rules: separation, alignment, cohesion (classic Boids).
 - Agents avoid obstacles and follow paths.
 - User can interact (click to attract or repel the flock).
 - Visualization options (toggle trails, change colors).

Project Rubric (Total = 15)

Criteria	Points	Description
1. Project Requirements & Functionality	5	All required features are implemented and working correctly. Includes interactivity, animations, and overall application goals.
2. Code Quality & Organization	2	Code is clean, modular, uses functions/classes, and is easy to follow with meaningful variable names and no major duplication.
3. Creativity & Design	2	Project shows originality, good visuals, engaging animations/interactions, and thoughtful design choices.
4. Performance & Stability	3	Runs smoothly without major bugs or crashes; animations and interactions are responsive.
5. Documentation & Readme	3	Code is well-commented, README file clearly explains how to run, use, and understand the project. PDF includes illustrative code snippets with example output visuals.

.Example README.md

Interactive Storybook Project

This project is an interactive storybook built in Processing.
It has multiple scenes, animated characters, and branching story paths.

Features

- 3+ scenes with animated transitions
- Two characters per scene
- User choices change the story direction
- Simple sound effects for transitions

How to Run

1. Open `Storybook.pde` in the Processing IDE.
2. Click the ****Run**** button (▶).
3. Follow the on-screen instructions to interact with the story.

Controls

- Mouse click: Choose the story path
- `R`: Reset the story
- `S`: Toggle sound on/off

Example Output

![screenshot](screenshot.png)

.....

*my
Best
Wishes!*

